Recall and F1-Score Metrics

1. What You'll Learn in This Section

In this lesson, you'll learn to:

Calculate precision and recall from a confusion matrix and explain what each one tells you.
Explain how raising or lowering the classification threshold trades precision against recall.
Calculate F1-score as the harmonic mean of precision and recall and explain why it exposes a lopsided model that a simple average would hide.
Decide whether a real-world classification system should be tuned for high precision, high recall, or balance, based on the domain.

2. Detailed Explanation

Confusion matrix recap: TP, TN, FP, FN

A confusion matrix compares what a model predicted against what actually happened, for a two-class (positive/negative) problem. It makes clear exactly where a model gets "confused" — if a large share of actual class-one instances get predicted as class two, the model is confused about class one and needs attention.

Every prediction falls into one of four buckets:

Outcome	Meaning
True Positive (TP)	Correct prediction — actually positive, predicted positive.
True Negative (TN)	Correct prediction — actually negative, predicted negative.
False Positive (FP)	Actual negative wrongly predicted positive — the Type 1 error.
False Negative (FN)	Actual positive wrongly predicted negative — the Type 2 error.

A trustworthy model produces high TP and TN counts and low FP and FN counts. Heavy confusion on a test set signals the model needs more work before release.

Precision, covered here as a recap before recall, measures how many of the predicted-positive instances are actually positive:

```
Precision = TP / (TP + FP)
```

Precision only looks at the subset of instances the model flagged positive. Imagine a dataset with 20 actual positive instances where the model predicts 10 instances as positive, 6 correctly (TP = 6) and 4 incorrectly (FP = 4). Precision describes only those 10 predicted-positive instances — it says nothing about the actual positives the model never flagged at all, which become false negatives. That's why precision alone is an incomplete metric: it cannot tell you what percentage of the true positive population the model actually captured.

Recall (sensitivity)

Recall answers the question precision cannot: of all the actual positive instances in the dataset, how many did the model correctly identify?

```
Recall = TP / (TP + FN)
```

Recall is also known as sensitivity — how sensitive the model is to actual positive data points, i.e., whether it captures most of them or misses many.

Think of it this way:

> Precision asks: "Can I trust a positive call?" — with what confidence can predicted positives be trusted.
> Recall asks: "Did we catch everything we need to?" — did the model capture all the actual positives it was supposed to find.

Consider a defective-versus-genuine-product example, where defective is the positive class because it's the class of interest (even though the genuine/good class matters equally in practice). If a dataset has 20 actually defective products and the model correctly flags only 10 of them, the rest go undetected, and recall is 10/20 = 0.5.

Precision and recall are complementary, not interchangeable — they surface different information, and neither replaces the other. A good system needs to be high on both: trustworthy about what it flags as positive (high precision), and not leaving real positives undetected (high recall).

Why a single combined metric is needed

When comparing two models trained on the same dataset (for example, logistic regression versus a support vector machine), you need one comparable score. If one model wins on precision and the other wins on recall, precision and recall alone can't declare a winner.

Accuracy could serve as that single score, but accuracy is unreliable when class distribution is imbalanced — it hides poor performance on the minority class, especially when the positive class (the class of interest)

has very few instances. F1-score was introduced to fill this gap: a single metric that combines precision and recall.

Case study: forest fire risk prediction

A network of forest sensors collects images, temperature, humidity, and wind level, and a model predicts whether a region is at fire risk. If fire risk is predicted, an alarm is raised, an evacuation team is sent, and safety measures begin; if not, resources are saved. Because manual inspection and local reporting are often unreliable, government agencies increasingly turn to AI-based monitoring. This is a binary classification problem, solvable with an algorithm such as logistic regression, and fire risk is the positive class because more action is required when it occurs.

For this problem:

> **True Positive**: predicted fire risk, region actually at fire risk.
> **True Negative**: predicted no fire risk, region actually safe.
> **False Positive**: predicted fire risk, region actually safe (Type 1 error).
> **False Negative**: predicted no fire risk, region actually had fire risk (Type 2 error).

Base numbers — 25 forest regions:

Actual \ Predicted	Predicted Safe	Predicted Fire
Actual Safe	13 (TN)	2 (FP)
Actual Fire	4 (FN)	6 (TP)

- **Precision** = TP / (TP + FP) = 6 / 8 = 0.75 — 75% of the instances predicted as fire risk were genuinely at fire risk (25% were incorrect).
- **Recall** = TP / (TP + FN) = 6 / 10 = 0.6 — only 60% of the actual fire-risk instances were correctly caught; 40% were missed.

Watch out for: the position of each cell inside a confusion matrix isn't fixed by convention — it depends on which class is listed first in the rows and columns. If "safe" is listed first instead of "fire," the cell order changes. Always read the layout against the actual row/column labels rather than memorizing a fixed position.

Threshold and the precision-recall trade-off

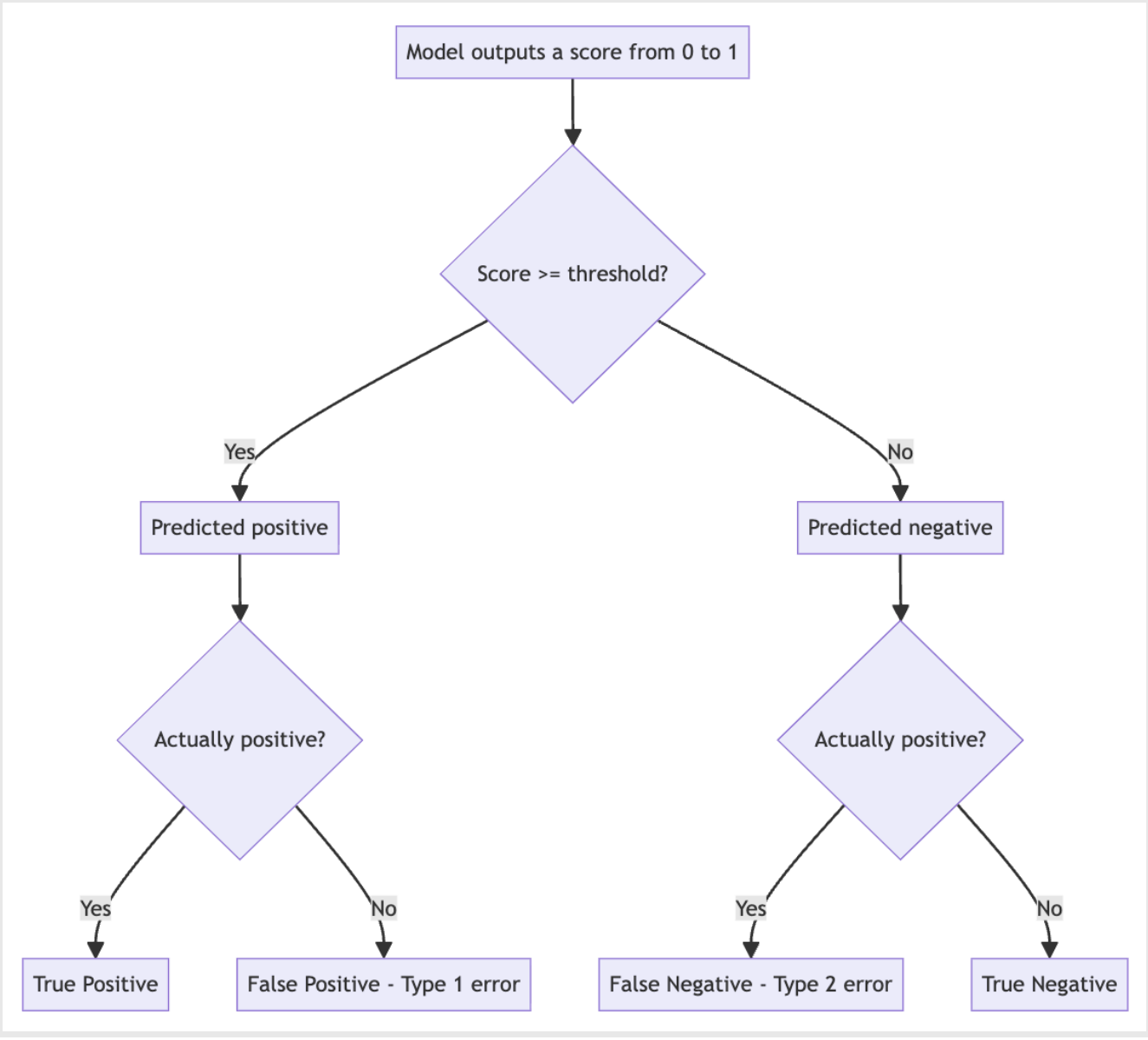
A classification model outputs a score between 0 and 1. Any instance scoring at or above the threshold is predicted positive; below it, negative. The default threshold is 0.5.

- **Raising the threshold** (e.g., to 0.6 or 0.7) makes the model more conservative — only very confident predictions get flagged, so false positives shrink and precision rises. But boundary-case

instances that were genuinely positive, yet scored just below the new threshold, get missed — false negatives increase and recall drops.

- **Lowering the threshold** (e.g., to 0.3) makes the model flag more instances as positive, including boundary cases. False negatives shrink and recall rises, but more actual negatives get swept in as false positives, so precision drops.

The 0.5 default is a standard middle-ground threshold because it favors neither metric. On a graph with threshold on the x-axis and precision/recall on the y-axis, precision rises and recall falls as threshold moves right. The two curves cross near the middle, close to 0.5, which is also where F1-score peaks. Moving away from that crossover point in either direction degrades F1, because one of the two metrics degrades sharply. This trade-off is a common interview topic: given a threshold shift from 0.5, you should be able to state what happens to precision, recall, and F1.



F1-score: the harmonic mean

Since precision and recall each measure something different, you need a single measure to compare models directly, the way accuracy would if data were balanced.

Why a simple average is misleading. Taking the arithmetic mean of precision and recall (e.g., (0.75 + 0.6) / 2 = 0.675) uses addition. Addition can't signal that one value is significantly worse than the other, because swapping which value is high and which is low doesn't change the sum. A model very strong on one metric and very weak on the other can still produce a deceptively decent-looking average.

F1 is the harmonic mean of precision and recall:

```
F1 = 2 * Precision * Recall / (Precision + Recall)
```

Because precision and recall are multiplied in the numerator, if either one is low, F1 gets pulled down noticeably; if both are low, it drops further. The harmonic mean always leans toward the smaller of the two inputs, so F1 exposes weaknesses a simple average would mask.

- **Fire dataset**: Precision = 0.75, Recall = 0.6 → F1 = 2 × 0.75 × 0.6 / (0.75 + 0.6) = 0.667. F1 sits closer to recall (0.6) than to precision (0.75), correctly signaling that recall is the model's weaker side — it's missing a meaningful share of actual fire-risk cases even though precision looks strong.
- **High-precision, low-recall example**. Raising the threshold on the same 25-region scenario can push TP to 4, FN to 6, FP to 0 (out of 10 actual fire-risk instances). Precision = 4/4 = 1.0, Recall = 4/10 = 0.4. Precision alone looks perfect, but only 4 of 10 real fire-risk cases were caught.
- **Extreme imbalance example**. Precision = 0.15, Recall = 0.95: the simple average is (0.15 + 0.95) / 2 = 0.55, suggesting "55% good." The actual F1-score is 2 × 0.15 × 0.95 / (0.15 + 0.95) = 0.26, revealing the system is really reliable only about 26% of the time. The simple average hides this because it doesn't penalize a large imbalance between the two metrics the way the harmonic mean does — the same reason accuracy misleads on imbalanced datasets. F1 has remained the standard for classification problems for years for exactly this reason.

Comparing biased vs. balanced models

Three variants of the same underlying model, each tuned to a different threshold, illustrate the trade-off on a bar chart of recall, precision, and F1:

- **A precision-focused model (moderate)**: precision pushed higher by raising the threshold slightly (precision around 0.90, recall around 0.53), but F1 comes out to only 0.67 — the weaker metric (recall) drags the combined score down.
- **An extreme-precision-focused model**: precision pushed even closer to 1.0 while recall drops to around 0.4, giving an F1 of only 0.57 — F1 exposes that the model isn't well-rounded despite the very strong precision number.
- **A balanced model**: precision and recall both kept reasonably high and close together (around 0.78 precision, 0.7 recall), producing the highest F1 of the three (roughly 0.7-0.74) — clearly better overall than either precision-biased model.

The pattern holds generally: even if one of precision or recall is very high, F1 will only be high when **both** are simultaneously high or acceptable; a lopsided model is penalized. This is why the threshold is usually kept near the default middle value (0.5) when the goal is an overall-good, balanced system rather than one optimized for a single metric.

Choosing between precision and recall: domain trade-offs

Whether a system should be tuned for high precision, high recall, or balance is a domain decision made by whoever benefits from the model, not a fixed rule.

Medical/cancer diagnosis example. In busy, high-demand hospitals — such as a major cancer hospital in Mumbai or a government hospital in Delhi — doctors face far more patients than they can immediately treat. Lower-risk or early-stage cases are managed with medicine and postponed rather than operated on immediately. Here, the model benefits from being tuned for **high precision:** if the system flags a case with a high classification score, that case must be genuinely malignant. It then escalated quickly to a senior doctor for immediate treatment. Junior doctors handle lower-scoring cases and only escalate once a case becomes serious, since senior doctors are a scarce, reserved resource.

Defective-product identification example. Here the goal is to catch essentially every defective product, since a missed defect that reaches customers damages the company's reputation. This favors **high recall**: lowering the threshold so that all actual defective products are flagged, even at the cost of extra false positives. For example, with 25 actually defective products out of 500 total, tuning for high recall to catch all 25 might also pull in 10 extra false positives, for 35 total items sent to manual inspection. That's an acceptable trade-off, since manual review of extra items is manageable, but missing a real defect is not.

The precision/recall trade-off decision is domain- and stakeholder-driven and can be genuinely debatable. A developer's job is to understand the direction and consequences of the trade-off, not to assume one universal answer.

Multi-class classification

For more than two classes, the confusion matrix idea still applies, but precision, recall, and F1 are computed **per class** — precision for class one, class two, class three, and so on, with recall and F1 computed the same way for each class. An overall score is then obtained by averaging across classes.

Code walkthrough: computing metrics with pandas and scikit-learn

The fire-risk case study was rebuilt as a pandas DataFrame, then the metrics were recomputed with library functions to confirm they match the manual calculations.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, recall_score,
f1_score
from sklearn.linear_model import LogisticRegression
```

The 25-region dataset is built with an `actual` column of 10 rows of 1 (fire present: 6 to be matched as TP, 4 to become FN) followed by 15 rows of 0 (safe: 13 to be matched as TN, 2 to become FP), with a `predicted` column constructed to line up against it and reproduce TP = 6, FN = 4, TN = 13, FP = 2.

The confusion matrix is computed with the negative class ("safe" = 0) listed first and the positive class ("fire" = 1) listed second — a deliberate choice, since cell ordering must be read from the labels used, not assumed:

```
cm = confusion_matrix(demo["actual"], demo["predicted"], labels=[0, 1])
tn, fp, fn, tp = cm.ravel()
print("TN:", tn, "FP:", fp, "FN:", fn, "TP:", tp)  # TN=13, FP=2, FN=4, TP=6

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["safe", "fire"])
disp.plot(cmap="YlOrRd", colorbar=False)
plt.title("Confusion Matrix with 25 Monitored Regions")
plt.show()
```

`ravel()` flattens the 2x2 confusion matrix array into a single one-dimensional array of four values in row-major order.

Manual and library recall are compared to confirm `recall_score` isn't a "black box" — it's the same TP/(TP+FN) computation:

```
recall_manual = tp / (tp + fn)
print(f"Recall: {recall_manual:.3f}")  # 0.600

recall_sklearn = recall_score(demo["actual"], demo["predicted"])
print(np.isclose(recall_manual, recall_sklearn))  # True
```

In practice, use the library function directly rather than recomputing manually each time. The same extreme-precision scenario (FP = 0, TP = 4, FN = 6) is reproduced in code with `confusion_matrix` and `recall_score`, giving precision = 4/4 = 1.0 and recall = 0.4 — confirming that forcing precision to 100% by tightening the threshold drives recall down sharply.

A manual F1 function, with a zero-guard for the case where no instance is predicted positive at all (which would otherwise divide by zero):

```
def f1_from_precision_recall(precision, recall):
    if precision == 0 and recall == 0:
        return 0
    return 2 * precision * recall / (precision + recall)
```

Applying it to the fire dataset gives F1 = 0.667 (manual), matching `f1_score(demo["actual"], demo["predicted"])` from scikit-learn. The extreme-imbalance case (precision = 0.15, recall = 0.95) again gives simple average 0.55 versus the harmonic-mean F1 of 0.26.

A three-model comparison feeds three synthetic confusion-matrix configurations (TP, FN, FP, TN) through a helper function that returns recall, precision, and F1 via `recall_score` and `f1_score`:

- Recall-focused model: TP = 9, FN = 1, FP raised, TN reduced — false negatives pushed toward zero by aggressively predicting positive, which raises false positives as a side effect. F1 = 0.667.
- Precision-focused model: tuned the opposite way (fewer false positives at the cost of more false negatives). F1 = 0.57.

- Balanced model: TP = 7, FN = 3, FP = 2, TN = 13 — neither error type driven to zero. Recall = 0.7, Precision = 0.778, F1 = 0.737 — the strongest overall of the three.

The results are assembled into a small DataFrame indexed by model name, with recall, precision, and F1 as columns, plotted as a grouped bar chart using `tight_layout()` to keep bar labels from overlapping.

A closing cell loads the scikit-learn breast cancer dataset, fits a `LogisticRegression` model, and loops over threshold values (0.1, 0.3, 0.5, and so on) to print precision, recall, F1, and the number of positive-flagged instances at each threshold — reproducing the same trade-off pattern on a real dataset. This cell is introduced but not run in depth, since it depends on preprocessing steps such as `StandardScaler`'s `fit_transform` that go beyond this material.

What machine learning is

Machine learning is a set of techniques, derived from mathematics and statistics, used mainly for two purposes: **classification** and **regression**. Its algorithms include **logistic regression, linear regression, support vector machines, neural networks, and transformers**. Its foundation is purely mathematical rather than rooted in computer science or mechanical/civil engineering, but these algorithms are implemented and run on computers, so they're studied both theoretically and through code. In a data-science context, machine learning is applied to data from any domain of interest — numeric, mechanical, financial, and so on — to infer information from that data. Data science is essentially the application of machine learning algorithms to a domain's data.

For further study, a machine learning book by **Christopher** Manning (Stanford), framed around natural language processing but covering general algorithms, is recommended as thorough and clear. His *Foundations of Statistical Natural Language Processing* is highlighted as especially effective for text datasets. Books by **Tom Mitchell** (CMU, with a free PDF available and accompanying videos) and by **Andrew Ng** (alongside his well-known Coursera course) round out the recommended reading.

3. Key Takeaways

Precision (TP/(TP+FP)) tells you how trustworthy a positive prediction is; recall (TP/(TP+FN)), also called sensitivity, tells you how much of the true positive population the model captured.
False positives are Type 1 errors; false negatives are Type 2 errors.
Raising the classification threshold above the 0.5 default increases precision and decreases recall; lowering it below 0.5 does the opposite.
F1-score, the harmonic mean of precision and recall, leans toward the weaker of the two metrics and exposes lopsided models that a simple average would hide.
Whether to optimize for high precision, high recall, or balance is a domain- and stakeholder-driven decision — precision-first in overloaded medical triage, recall-first in defective-product detection.

Mental model: Think of precision as a strict gatekeeper — it only judges the people it let in — and recall as a search party that's graded on how many of the missing people it actually found. F1-score is the referee that refuses to award a good grade unless both are doing their job well.